

3 GETTING FAMILIAR WITH PYTHON

Before start writing extensive Python scripts, this chapter gently helps you to understand the importance of basic Python programming components. It starts with the explanation and limitations of a pre-defined set of words called **keywords**. Although we have made this discussion very limited about the keywords you will get to know more about them according to their need in later chapters.

After that, we have explained two other programming tools that are important for documentation purposes such as **Naming Conventions** for identifiers and **Commenting** a Python script. Mostly these two tools are required by the Python developer which improves code readability and acceptance among other developers.

The next section describes **Python Types** deeply discussed various data types also known as Objects either as pre-defined or created with their uses. Later in the Sections **Object Types, Input and Output** and **Variable Assignments** discussed how to use these types either with directly assigned values or by getting values from the user as input and producing output by manipulating them.

Finally, the last Section highlights **Built-in Functions** comes with python interpreter. In this chapter based on the necessity, we have also imported some of the modules in code examples using import statement. But we have not discussed much about importing a module in this Chapter, rather we keep it separately in Chapter 8.

3.1 Keywords

Every programming language has special reserved words, or keywords, that have specific meanings and restrictions around how they should be used. Python is no different. Python keywords are the fundamental building blocks of any Python program.

Python keywords are special reserved words that have specific meanings and purposes and can't be used for anything but those specific purposes. These keywords are always available—you'll never have to import them into your code.

Python keywords are different from Python's built-in functions and types. The built-in functions and types are also always available, but they aren't as restrictive as the keywords in their usage.

An example of something you can't do with Python keywords is assign something to them. If you try, then you'll get a `SyntaxError`. You won't get `SyntaxError`, if you try to assign something to a built-in function or type, but it still isn't a good idea. As of Python 3.10, there are thirty-five keywords in Python. A list of keywords is given in Table 3.1.

Table 3.1 List of Keywords in Python 3.10

False	Class	from	or
None	Continue	global	pass
True	Def	If	raise
And	Del	import	return
As	Elif	In	try
assert	else	Is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	For	not	

The list of keywords is also retrieved using help in-built functions by passing 'keywords' as a parameter to it. Consider the code snippet given below:

```
>>>help('keywords')
```

`help()` function can be used to obtain more information about any keyword, by passing that keyword as argument to it, as follows:

```
>>> help('pass')
```

```
The "pass" statement
*****

    pass_stmt ::= "pass"

"pass" is a null operation — when it is executed, nothing happens. It
is useful as a placeholder when a statement is required syntactically,
but no code needs to be executed, for example:

    def f(arg): pass      # a function that does nothing (yet)

    class C: pass        # a class with no methods (yet)
```

Python offers a module namely keyword to check a given string is a keyword or soft keyword (added in Python 3.10). Soft keywords are only reserved for specific contexts and identifiers **match**, **case** and **_** (underscored sign) can syntactically act as a keyword's context related to pattern matching. Table 3.2 highlights the details of keyword module.

Table 3.2 Methods and Constants defines in the keyword module

Methods	and	Description
Constants		
iskeyword(s)		Return True if <i>s</i> is a Python keyword.
kwlist		Return a sequence containing all the keywords defined for the interpreter.
issoftkeyword(s)		Return True if <i>s</i> is a Python soft keyword.
softkwlist		Return a sequence containing all the soft keywords defined for the interpreter.

The import statement is necessary to use any of these methods and constants. See the code sample

given below:

```
>>>import keyword
>>>keyword.softkwlist
['_', 'case', 'match']
>>>keyword.iskeyword('is')
True
>>>keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
>>>keyword.issoftkeyword('match')
True
```

3.2 Naming Convention

These are set of rules for choosing the character sequence to be used for the naming of identifiers such as variables, constants, types, functions, and other entities in source code and documentation. For the following reasons naming conventions are followed by the programmers:

- to reduce the effort to understand the source code and to avoid naming collisions of the identifiers.
- to enable code reviews to focus on issues rather than focusing on the syntax and naming standards.
- to provide additional information (i.e., metadata) about the use to which an identifier is put.
- to promote consistency within a development team, so that each one can understand other's code.
- to enable the use of automated refactoring or search and replace tools with minimal potential for error handling.
- to provide a better understanding in case of code reuse after a long interval of time.
- In Python, a variable can have a short name (like *x* and *y*) or a more descriptive name such as **age**, **carname**, **total_volume**. A Python developer needs to follow the following rules as a set of naming conventions according to the coding standards:
 - A name must be start with an alphabet (A-Z, a-z) or the underscore (_);
 - name can't start with a number (0-9);
 - names can be alpha-numeric but strictly start with an alphabet (A-Z, a-z) or the underscore (_) only.
 - names are case-sensitive in Python, which means age, Age, and AGE represent three different identifiers.

Listing 3.1 highlights a couple of examples of valid identifier names and some of the invalid names are listed in Listing 3.2 and using any of these will raise `SyntaxError` by the interpreter.

Listing 3.1 Examples of valid identifier names

```
>>>lan = 'Python'
>>>ver_1 = 3.7
>>>present_ver = 3.11
>>>_ = 3
>>>_IDE = 'idle'
>>>Vers = [2.7,3.7,3.11]
```

Listing 3.2 Examples of invalid identifiers names

```
>>>my name = 'Goutam'
>>>my-name = 'Goutam'
>>>#marks = 3
>>>marks# = 3
>>>1_address = 'VIT'
>>>1 address = 'VIT'
```

Sometimes it is necessary that an identifier needs a descriptive name having multiple words. There are several techniques you can use to make them more readable:

- **Camel Case:** each word, except the first, must be starts with an uppercase letter (A-Z) only, such as myFirstName, presentAddress;
- **Pascal Case:** each word must be starts with an uppercase letter such as MyFirstName, PresentAddress;
- **Snake Case:** each word is separated by an underscore (_) character such as first_name, present_address;

3.3 Comments

When writing code, it's important to make sure that your code can be easily understood by others. Giving variables obvious names, defining explicit functions, and organizing your code are all great ways to do this. Another awesome and easy way to increase the readability of your code is by using comments!

Comments are an integral part of any program. In Python, comments come in the form of module-level docstrings, or even inline explanations that help shed light on a complex function. Before diving into the different types of comments, let's take a closer look at why commenting on your code is so important.

First, this section highlights two possible scenarios in which a programmer decided not to comment on their code. So that you can understand the importance of proper documentation of Python code and later you are going to know how to comment on Python scripts.

▪ **Reading your own code** - Client A wants a last-minute deployment for their web service. You're already on a tight deadline, so you decide to just make it work. All that “extra” stuff—documentation, proper commenting, and so forth—you'll add that later.

The deadline comes, and you deploy the service right on time and you have noted and go back to update the comments, but before you can put it on your to-do list, your boss comes over with a new project that you need to get started on immediately. Within a few days, you'd completely forgotten that you were supposed to go back and properly comment on the code you wrote for Client A.

Suppose after six months Client A needs a patch built for that same service to comply with some new requirements. It's your job to maintain it since you were the one who built it in the first place. Now it will be difficult to recall all the developments that you have made and after spending hours parsing through your old code, you're completely lost.

You were in such a rush at the time that you didn't name your variables properly or even set your functions up in the proper control flow. Worst of all, you don't have any comments in the script to tell you what's what! Developers forget what their code does all the time, especially if it was written a long time ago or under a lot of pressure. When a deadline is fast approaching, and hours in front of the computer have led to bloodshot eyes and cramped hands, that pressure can be reflected in the form of code that is messier than usual. Writing comments as you go is a great way to prevent the above scenario from happening. Be nice to Future You!

▪ **Your code is being read by others** - Imagine you're the only developer working on a small Django project. You understand your own code well, so you don't tend to use comments or any other sort of documentation, and you like it that way. Comments take time to write and maintain, and you just don't see the point.

The only problem is, by the end of the year your “small Django project” has turned into a “20,000 lines of code” project, and your supervisor is bringing on additional developers to help maintain it. The new developers work hard to quickly get up to speed, but within the first few days of working together, you've realized that they're having some trouble. You used some unorthodox variable names and wrote with super short syntax. The new developer spends a lot of time stepping through your code line by line, trying to figure out how it all works. It takes a few days before they can even help you maintain it!

Using comments throughout your code can help other developers in situations like this one. Comments help other developers skim through your code and gain an understanding of how it all works very quickly.

Now that you understand why it's so important to comment on your code, let's go over some basics so you know how to do it properly. Comments are for developers. They describe parts of the code where necessary to facilitate the understanding of programmers, including yourself. A comment can be any of these two types:

3.3.1 Single line commenting

We can do this by putting a hash mark (#) before your desired comment. Consider the example given below:

```
>>>#This is a comment
>>>print('Commenting in python')
```

Commenting in python

Python interpreter doesn't interpret a line of code if the line is preceded with a # till the end of the line. That's why we didn't see anything in the output from the first line of the code.

Note: Comments should be short, sweet, and to the point. While PEP 8 advises keeping code at 79 characters or fewer per line, it suggests a max of 72 characters for inline comments and docstrings. If your comment is approaching or exceeding that length, then you'll want to spread it out over multiple lines.

3.3.2 Multiline Commenting

Python doesn't have a way to write multiline comments as you can in other languages such as C, Java, and Go. Don't try this with your code, or else you will end with a `SyntaxError`.

```
>>># So you can't
just do this
in python
```

`SyntaxError: invalid syntax`

For the above code interpreter only ignores the first line as single line comment and it interprets as wrong syntax for the others and raised a `SyntaxError`. While Python doesn't have native multiline commenting functionality, you can create multiline comments in Python in two ways. The first way is simply by pressing the return key after each line, adding a # symbol, and continuing your comment from there. See the example given below:

```
>>># This is a pretty good example
>>># of how you can spread comments
>>># over multiple lines in Python
```

The second approach is embedding comments in your code as multiline strings by wrapping your comment inside a set of triple quotes, as follows:

```
>>>"""
If I really hate pressing enter and
typing all those hash marks, I could
just do this instead
"""
```

While this gives you multiline functionality, this isn't technically a comment. It's a string that's not assigned to any variable, so it's not called or referenced by your program. Still, since it'll be ignored at runtime and won't appear in the bytecode, it can effectively act as a comment.

Note: Sometimes it also depending on where we are putting these three quotes ("""---""") in the code. If we put it inside the function definition it will be the part of Python byte code. This one is known as docstring.

3.4 Python Types

In Python data types take in the form of objects – where some of them comes as in-built with Python or some of them we can create using classes. In this Section, the discussion mainly focuses on built-in types that come with Python.

Before we start with the discussion let's see where data types fall in Python. From a real-world point of view, every information that can be processed and stored is known as data. Such as your name, address, contact details, age, etc. and Python provides certain types of built-in objects used to hold the meaningful information as identifier.

If you are coming from other programming domain such as C, C++ or Java, you will be responsible to take care the types of every data, which means you must decide it in advance, what will be the type of data an identifier can hold throughout its lifetime. But due to the dynamic nature of Python most of this work goes away, which means an identifier can hold a value of any type at any point of execution. Table 3.3 highlights Python built-in types with their class name.

Table 3.3 Python built-in Object types

Object Type	Data Type
Numeric	int, float, complex
Numeric with user-defined precision	Fraction for rational and Decimal for floating point numbers
Boolean	bool for truth values
None	NoneType holding no values
Iterator	Can be used with any of the sequence types
Sequence types	list, tuple, range
Set	set, frozenset
Mapping type	dict called dictionary as pair of key and value
Text Sequence type	str known as string
Binary Sequence types	bytes, bytearray, memoryview

In Table 3.3 three classes are categorized as sequence type, but sets, dictionary, and string can also be considered as sequence type objects in python. It means values holding by each sequence types can be accessed sequentially.

The class names or data types are highlighted with bold text in Table 3.3 and in next three sub-sections numeric types are discussed broadly in Section Numeric Type and all sequence types are formally introduced in Section Sequence Type. This discussion on strings is also limited to the introductory part only, which is broadly discussed separately along with its various operations.

3.4.1 Numeric Type

Python has three numeric data types such as **int** as integer, **float** as floating-point, and **complex** for complex numbers. In addition, Booleans are also considered as subtypes of integers, supporting only two values True and False. Integers don't have any fractional part; floating-point numbers are designed to work with real numbers and complex numbers have two parts namely **real** and imaginary (as **img**) and individually both are floating-point types. Later in Python 2.4 and 2.6 introduces two other numeric types as Decimal and Fraction classes. Decimal type is introduced with fixed precision and Fractions are implemented on rational objects.

Integers

In Python integers are whole numbers i.e., no fractions, no decimal points, nothing fancy. Well, aside from a possible initial sign and bases, if you want to express numbers in other ways than the usual decimal (base 10). The `int()` constructor method can be used to create an integer object or directly a number with possible sign can be assigned. Any sequence of digits in Python represents an integer literal and examples of all valid integer literal forms are given in code Listing 3.3:

Listing 3.3 Example of integers of decimal base

```
>>>a = +5
>>>b = 5
>>>c = -5
>>>d = 1_00_000
>>>e = int(4)
```

A negative integer number should be preceded with a negative (-) sign, and numbers with optional plus (+) sign are considered positive integers as assigned to the variable **a** (with sign) and **b** (without sign). Integers are assumed to be decimal (base 10) unless prefix characters are used to specify another base. You might never need to use these bases, but you'll probably see them in Python code somewhere, sometime. In Python, you can express literal integers in three bases besides decimal with these integer prefixes as: (a) 0b or 0B for binary (base 2); (b) 0o or 0O for octal (base 8); and (c) 0x or 0X for hexadecimal (base 16).

These bases are all powers of two (2), and are handy in some cases, although you may never need to use anything other than good old decimal integers as shown in code Listing 3.3. Check the code Listing 3.4 to know how to assign values in other integer form using prefix characters.

Listing 3.4 Examples of other Integer types

```
>>>a = 0b11
>>>b = 0B110
>>>c = 0o245
>>>d = 00345
>>>e = 0x2345a
>>>f = 0X34af
>>>print(a, b, c, d, e, f)
3 6 165 229 144474 13487
```

The prefix characters are supported in upper (0B, 0o, or 0x) as well as lower (0b, 0O, or 0X) case. We must note that while using any integer to an expression for example in `print()` statement at line 7 values will be always processed in decimal form, as values are printed in the output of code Listing 3.4. Pythons have three in-built function to do that. `bin()`, `oct()`, and `hex()` are used to convert the decimal form of integer to binary, octal and hexa-decimal respectively.

```
>>> print(bin(a), oct(c), hex(e))
0b11 0o245 0x2345a
```

Python 3.x supports unlimited precessions means any size of integer:

[illegible]

Unlimited precision integers are built-in tools in Python and support up to memory of your device. They are also why we can raise large exponent numbers:

```
>>>45 ** 40
1344313472276061687044450174711376326749814325012266635894775390625
```

Floating-Point

In Python real numbers are represented by the class float with fixed 8 bytes (or 64 bits) of memory. These 64-bits are aligned into three different parts, as (a) a single bit is reserved as sign (positive or negative) bit; (b) 11 bits are reserved for exponent $1.5e^{-5}$ 1.5×10^{-5} (exponent is -5) within a range of $[-1022, 1023]$; and (c) 52 bits are significant digits.

In computers floating-point numbers are represented in base 2 (binary) fractions. For example, the decimal fraction 0.125 has value $1/10 + 2/100 + 5/1000$, and in the same way the binary fraction 0.001 has value $0/2 + 0/4 + 1/8$. These two fractions have identical values, the only real difference being that the first is written in base 10 fractional notation, and the second in base 2. For example, 12.345 can be represented as a base-ten floating-point number:

$$12.45 = \underbrace{12345}_{\text{significand}} \times \underbrace{10}_{\text{base}} \overset{\text{exponent}}{\underbrace{-3}}$$

The constructor method `float()` of the `float` class returns a floating-point number based on the number or a string:

```
>>>float(3.45)
3.45
>>>float('3.45')
3.45
```

In computers there are very few fractional numbers represented as binary fractions, instead an

approximated binary floating-point number is stored into memory. Consider the fraction $1/3$, in decimal base 10 it can be 0.3 or 0.33 or 0.333 and so on. No matter how many digits you're willing to consider, the result will never be same. Let's do a comparison between two float numbers:

```
>>>x = 0.1 + 0.1 + 0.1
>>>y = 0.3
>>>x==y
False
```

It looks very strange, variables `x` and `y` having same value but comparison states different story. This is because Python cannot use a finite number of digits to represent the numbers. Let's check this using `format()` function:

```
>>>format(x, '.20f')
'0.30000000000000004441'
>>>format(y, '.20f')
'0.29999999999999998890'
```

Here we have taken upto 20 decimal place and exactly the numbers are not stored into computer memory while using binary decimal. Just remember, even though the printed result looks like the exact value of 0.3, the actual stored value is the nearest representable binary fraction.

Note: This is in the very nature of binary floating-point: this is not a bug in Python, and it is not a bug in your code either. You'll see the same kind of thing in all languages that support your hardware's floating-point arithmetic (although some languages may not display the difference by default, or in all output modes).

PEP485²⁰ provides a solution that fixes this problem by using relative and absolute tolerances. It provides `isclose()` function of `math` module and returns `True` if two numbers are relatively close to each other:

```
>>>x = 0.1 + 0.1 + 0.1
>>>y = 0.3
>>>from math import isclose
>>>isclose(x,y)
True
```

Some of these drawbacks are handled using `Decimal` and `Fraction` data types are discussed in the next two sections.

Decimal

Decimals are like floating-point numbers with fixed precisions. It was introduced in Python 2.4 to core numeric type as decimal object, formally known as `Decimal`. Like floating-point type we can't just assign any literals to a name, for that we need to import the decimal module. After all we need to answer why another numeric type like floating-point. Let's first list all the advantages of decimal type over floating-point type:

- Decimal numbers can be represented exactly. In contrast, numbers like 1.1 and 2.2 do not have exact representations in binary floating point. End users typically would not expect $1.1 + 2.2$ to display as 3.3000000000000003 as it does with binary floating point.

²⁰ <https://peps.python.org/pep-0485/>

- The exactness carries over into arithmetic. In decimal floating point, $0.1 + 0.1 + 0.1 - 0.3$ is exactly equal to zero. In binary floating point, the result is $5.5511151231257827\text{e-}017$, near to zero. For this reason, decimal is preferred in accounting applications that have strict equality invariants.
- The decimal module incorporates a notion of significant places so that $1.30 + 1.20$ is 2.50 . The trailing zero is kept indicating significance. This is the customary presentation for monetary applications. For multiplication, the “schoolbook” approach uses all the figures in the multiplicands. For instance, $1.3 * 1.2$ gives 1.56 while $1.30 * 1.20$ gives 1.5600 .

To create a decimal type need to import the decimal module first and need to go through the current context with `getcontext()` and if necessary we can set new values to its parameters:

```
>>>from decimal import *
>>>getcontext()
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1,
clamp=0, flags=[], traps=[InvalidOperation, DivisionByZero, Overflow])
```

The decimal number supports up to 28 decimal places and Python interpreter raises exception on getting signals of `InvalidOperation`, `DivisionByZero` and `Overflow`. The context for arithmetic is an environment specifying precision, rounding rules, limits on exponents, flags indicating the results of operations, and trap enablers which determine whether signals are treated as exceptions. Table 3.4 lists all the rounding options of decimal module:

Table 3.4 List of all rounding options of decimal type

Rounding options	Description
<code>decimal.ROUND_CEILING</code>	Round towards infinity
<code>decimal.Round_DOWN</code>	Round towards zero
<code>decimal.ROUND_FLOOR</code>	Round towards -infinity
<code>decimal.ROUND_HALF_DOWN</code>	Round to nearest with ties going towards zero.
<code>decimal.ROUND_HALF_EVEN</code>	Round to nearest with ties going to nearest even integer.
<code>decimal.ROUND_HALF_UP</code>	Round to nearest with ties going away from zero.
<code>decimal.ROUND_UP</code>	Round away from zero
<code>decimal.ROUND_05UP</code>	Round away from zero if the last digit after rounding towards zero would have been 0 or 5; otherwise round towards zero.

Signals are groups of exceptional conditions arising during computation. Depending on the needs of the application, signals may be ignored, considered as informational, or treated as exceptions. The signals in the decimal module are: `Clamped`, `InvalidOperation`, `DivisionByZero`, `Inexact`, `Rounded`, `Subnormal`, `Overflow`, `Underflow`, and `FloatOperation`.

A decimal number is immutable. It has a sign, coefficient digits, and an exponent. To preserve significance, the coefficient digits do not truncate trailing zeros. Decimals also include special values such as `Infinity`, `-Infinity`, and `NaN`. The standard also differentiates `-0` from `+0`. By calling `Decimal` constructor we can make a decimal object and passing the value as string with the desired number of decimal digits:

```
>>>import decimal
>>>decimal.Decimal('3.68')
Decimal('3.68')
```

On passing a floating-point value, it will take it up to the precision value of the current context:

```
>>>decimal.Decimal(3.68)
Decimal('3.6800000000000000159872115546022541821002960205078125')
```

Unlike hardware based binary floating point, the decimal module has a user alterable precision (defaulting to 28 places) and can be altered as follows:

```
>>>getcontext().prec=3
>>>Decimal('1')/Decimal('7')
Decimal('0.143')
```

If the FloatOperation signal is trapped, accidental mixing of decimals and floats in constructors or ordering comparisons raises an exception:

```
>>>getcontext().traps[FloatOperation] = True
>>>Decimal(5.7)
Traceback (most recent call last):
  File "<pyshell#68>", line 1, in <module>
    Decimal(5.7)
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
```

The significance of a new Decimal is determined solely by the number of digits input. Context precision and rounding only come into play during arithmetic operations only, not at the time of creation.

Listing 3.5 Use of context parameters in Decimal

```
>>>getcontext().prec = 6
>>>Decimal('3.0')
Decimal('3.0')
>>>Decimal('3.1415926535')
Decimal('3.1415926535')
>>>Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85987')
>>>getcontext().rounding = ROUND_UP
>>>Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85988')
```

In Listing 3.5, precision is set to 6, it doesn't make any impact on the next two Decimal creations. It considers all the digits those are passed to *Decimal()* constructor built-in function. But the addition operations of line 6 and 9 produces the result within the length of precision and the second expression rounding up the result up to 6th decimal place.

Fractional

The shortcoming of floating-point arithmetic in binary is address by Fraction data type. It's not coming as another built-in types like int or float rather we need import it from fractions module. If we need to work with financial data and require infinite precision for some calculation it will be preferred over floating-point type. Using Fraction class we can create many ways a fraction object depending on number of values that we are passing to **Fraction()** constructor method of fractions class. The class constructor accepts zero, one, or two arguments of various types:

Listing 3.6 Creation of Fraction Objects

```
>>>import fractions
>>>a = fractions.Fraction()
>>>b = fractions.Fraction(3)
>>>c = fractions.Fraction(3,4)
>>>print(a, b, c)
0 3 3/4
```

The first version in Listing 3.6 simply returns 0 as fractional number. The second variant of constructor takes 1 argument and returns 3 as Fractional number. The last variant takes two numbers as numerator and denominator. The third variant only accepts instances of rational number.

Note: To get the fractional value in human-friendly texture (with slash / character) it is necessary to `print()` the Fraction object.

For the third variant of `Decimal()` constructor, when we pass a valid numerator and denominator, they will be normalized if they have a common divisor:

```
>>>fractions.Fraction(3,6) #gcd(3,6) = 3
Fraction(1, 2)
>>>fractions.Fraction(0,5) #gcd(0,5) = 5
Fraction(0, 1)
```

The normalization also takes the minus sign into account when you define negative fractions:

```
>>>-fractions.Fraction(3,12)
Fraction(-1, 4)
>>>fractions.Fraction(-3,12)
Fraction(-1, 4)
>>>fractions.Fraction(3,-12)
Fraction(-1, 4)
```

The second variant constructor of Listing 3.6 also takes arguments of real and string types. From real types it accepts both floating-point and decimal number and both considered irrational number.

```
>>>print(fractions.Fraction(0.5))
1/2
>>>print(fractions.Fraction('3'))
3
```

For the floating-point type of result is the same in fractional notation. However, this code works as expected only by coincidence. In most cases, you won't get the intended value due to the representation error that affects float numbers, whether they're rational or not. For example, a fraction 1/10 can be accommodated as 0.1 in floating-point number. But in python story is different, because of binary representation of 0.1 stores an approximate value in memory.

```
>>>print(fractions.Fraction(0.1))
3602879701896397/36028797018963968
```

The previous number 0.5 can be accommodated in memory, so it gets the exact representation of fraction 1/2. But 1/10 can only be approximated with a non-terminating repeating expansion of binary digits. The numerator and denominator of a Fraction object can be access separately using numerator and denominator member of Fraction object:

```
>>>a = fractions.Fraction(3,4)
>>>print(a.numerator, a.denominator)
3 4
```

We have seen how to convert other data types to Fraction type. Converting between built-in data types in Python can be done by calling constructor methods `int()`, `float()`, `str()` and `complex()`. A fractional type can be directly converted to float type and internally it's done by dividing the numerator by denominator of the fraction.

```
>>>a = fractions.Fraction(3,4)
>>>float(a)
0.75
>>>str(a)
'3/4'
>>>complex(a)
(0.75+0j)
```

Fraction didn't provide any implementation directly to convert to integer. It simply truncates the fraction to smallest integer and like converting a float to an integer:

```
>>>a = fractions.Fraction(22,3)
>>>print(a)
>>>22/3
>>>int(a)    #truncates the integer part
7
```

Converting to Decimal type will raise a `TypeError`, since Fractions represents a division and to bypass this, we can convert any part as Decimal and dividing them manually:

```
>>>a = fractions.Fraction(12,3)
>>>d = a.numerator / Decimal(a.denominator)
>>>d
Decimal('4')
```

Choosing between Float, Decimal and Fraction

The float data type should be your default choice for representing real numbers in most situations. Floating-point numbers are mostly suitable for science, engineering, and computer graphics, where execution speed is more important than precision. Hardly any program requires higher precision than you can get with a floating-point anyway.

Floating-point stores values in binary float point and sometimes using the binary system doesn't provide enough precision for real numbers. One notable example is financial calculations, which involve dealing with very large and very small numbers at the same time. They also tend to repeat the same arithmetic operation repeatedly, which could accumulate a significant rounding error.

Real numbers can be stored using decimal floating-point arithmetic to mitigate these problems and eliminate the binary representation error. It's like a float as it moves the decimal point around to accommodate larger or smaller magnitudes. However, it operates in the decimal system instead of in the binary.

Both the Fraction and Decimal types share a few similarities. They address the binary representation error, they're implemented in software, and you can use them for monetary applications. Nevertheless,

Listing 3.8 Formation of complex number

```

>>>a = 4j
>>>b = 3j
>>>c = .2j
#Adding complex number with integer
>>>d = 3 + a
#Adding complex number with float
>>>e = 3.4 + b
>>>a, b, c, d, e
(4j, 3j, 0.2j, 3+4j, (3.4+3j))

```

It is also possible to access the real and imag part of a complex number separately using the real and imag attribute of a complex object:

```

>>>c = 4+6j
>>>c.real, c.imag
(4.0, 6.0)

```

Note: For the type of consistency in Python each part of the complex number is considered as float, because absolute value of a complex number is floating type. So, there is no reason to make a special case and return an **int**, if the output happens to be a round number. Also, note that the same reasoning applies for the components of your imaginary numbers which are always stored as float.

Boolean

Python 3 has another built-in type as Boolean, which has two values as True and False. Using built-in **type()** functions, it is possible to check the type of **True** and **False**:

```

>>>type(True)
<class 'bool'>
>>>type(False)
<class 'bool'>

```

An expression containing conditional, logical, identity and membership operators always evaluates to bool type. Details about working of Boolean type expression is discussed in Chapter 4. In Python built-in names are not keyword and we can assign them any value. But True and False are not built-in types, they are keywords and assigning any value to them will be raised a **SyntaxError**, as follows:

```

>>>bool = 45
>>>type(bool)
<class 'int'>
>>>a = True
>>>type(a)
<class 'bool'>
>>>True = 3

```

SyntaxError: cannot assign to True

Booleans are considered as numeric type in Python and any value other than 0 or **None**, will be treated as True. Because they are considered as numeric types, it is possible to perform arithmetic operations with them. At any point if you want to count the number of items that satisfy a condition, means number of True counts.

```

>>>bool(3)
True
>>>True + True
2
>>>int(True)
1
>>>bool(56.7)
True
>>>bool(0)
False
>>>bool(None)
False

```

Built-in function `len()` returns the length of a sequence type and in Python the length of an object considered as `True`:

```

>>>a = [3,4]
>>>len(a)
2
>>>bool(a)
True
>>>b = []
>>>len(b)
0
>>>bool(a)
False

```

The Python Boolean is a commonly used data type with many useful applications. You can use Booleans with operators like **not**, **and**, **or**, **in**, **is**, **==**, and **!=** to compare the values and check for membership, identity, or equality (discussed in Chapter 4). You can also use Boolean testing with **if** statement (discussed in Chapter 6) to control the flow of your programs based on the truthiness of an expression.

3.4.2 String

Discussion in this section is limited to string declaration only, details about string operations and methods can be found in Chapter 5. In Python strings are known as collection of characters and type of string is known as `str`. Strings literal is delimited using single or double quote and any number of characters between the opening and matching closing delimiter is considered as string:

```

>>>str1 = 'Hello'
>>>str2 = "Python"
>>>type(str1)
<class 'str'>

```

In Python a string can contain any number of characters that supports your system memory and it can be empty also by assigning a pair of single or double quotes or simply by calling `str()` constructor method of string class:

```
>>>str1 = ''
>>>str2 = str()
>>>str1, str2
('', '')
```

Any type can be converted to string by passing a value of type to `str()` constructor method inside quotes:

```
>>>str('1')
'1'
>>>str('4.6')
'4.6'
```

If you wish to include either type of quote character within the string, the simplest way is to delimit the string with the other type. If a string is to contain a single quote, delimit it with double quotes and vice versa:

```
>>>str1 = '"Contains double quotes"'
>>>str2 = '"Contains Single quotes'"
```

In Python a multiline string can be formed using any form of quotes three times. A multiline string can include escape sequences, but single quotes, double quotes, and newlines can be included without escaping them.

```
>>>str1 = '''This is a multiline
string.
Thats good'''
>>>print(str1)
This is a multiline
string.
Thats good
```

Strings are also categorized as a type of sequence means can be iterate each characters using a loop, which is covered separately in Chapter 5. Other built-in sequence types of lists, tuple, dictionary, sets are covered in the next Section.

3.4.3 Sequence Types

In Python sequence types supports a variety of operations as listed in Table 3.5 for both mutable and immutable type. In Table 3.5, s and t are sequences of the same type, n , i , j and k are integers and x is an arbitrary object that meets any type and value restrictions imposed by s . Details about these operations can be found in the respective chapter mentioned in 4th column of the table.

The **in** and **not in** operations have the same priorities as the comparison operations. The **+** (concatenation) and ***** (repetition) operations have the same priority as the corresponding numeric operations.

Table 3.5 List of operations supported by sequence types.

Operation	Operation Type	Result	Discussed in Chapter
$x \text{ in } s$	Membership	Return True if an item of s is x or False	4

<i>x not in s</i>	Membership	Return True if <i>x</i> is not an item of <i>s</i>	4
<i>s + t</i>	Concatenation	Appending <i>t</i> at the end of <i>s</i>	4, 5, 7
<i>s * n</i> or <i>n * s</i>	Addition	Equivalent to adding <i>s</i> to itself <i>n</i> times	4, 5, 7
<i>s[i]</i>	Indexing	<i>i</i> th item of <i>s</i>	5 and 7
<i>s[i:j]</i>	Slicing	Slice of <i>s</i> from <i>i</i> to <i>j</i>	5 and 7
<i>s[i:j:k]</i>	Slicing with Step	Slice of <i>s</i> from <i>i</i> to <i>j</i> with step <i>k</i>	5, 6, and 7
<i>len(s)</i>	size or length	Length of <i>s</i>	5, 6 and 7
<i>min(s)</i>	Minimum	Smallest item of <i>s</i>	7
<i>max(s)</i>	Maximum	Largest item of <i>s</i>	7
<i>s.index(x)</i>	Searching	Index of first occurrence of <i>x</i> in <i>s</i>	5 and 7
<i>s.count(x)</i>	Counting	No. of times <i>x</i> in <i>s</i>	5, 6 and 7

List

List is one of the built-in data structures in Python. It can accommodate multiple values of similar or dissimilar types. If you are coming from other programming domain, Python list is like the Vectors in C++ or ArrayList in JAVA. Lists are dynamic in nature, can grow or shrink through its insertion and deletion operation. List also supports searching for an item using membership operator or sequentially using a loop. Inserting and deleting the element from the beginning of the list is considered as costliest operation as all the elements are needed to be shifted by one position. This Section is all about the introduction to list as a data type and its assignments, detailed explanation can be found in Chapter 7. We will begin with declaration of list and accessing of it:

```
>>>list1 = list()
>>>list2 = []
```

Like other built-in types an empty list can be created using constructor method **list()** or by assigning a pair of empty square brackets (**[]**). List can be created by passing other sequence types (see Listing 3.9) to the constructor including strings.

Listing 3.9 Creation of list using other objects

```
>>>list('abc')
['a', 'b', 'c']
>>>list(range(3))
[0, 1, 2]
>>>d = {2,3,4} #set
>>>list(d)
[2, 3, 4]
>>>s = (1,2,3) #tuple
>>>list(s)
[1, 2, 3]
>>>di = {'a':2, 'b':3} #dictionary
>>>list(di)
['a', 'b']
```

Numeric types aren't iterable, and trying to do so will raise a `TypeError`, as follows:

```
>>>list(11)
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    list(11)
TypeError: 'int' object is not iterable
```

A list can be created using by assigned comma separated similar or dissimilar values inside a pair of square brackets:

```
>>>a = [2,3,4] #similar types
>>>b = [2,5.6,7+3j,'Python'] #different types
```

Tuple

Tuples are immutable (means can't alter after creation) sequence, typically used to store collections of dissimilar data. Tuples are also used for cases where an immutable sequence of homogeneous data is needed. Number of ways tuple can be created (see code Listing 3.10):

Listing 3.10 Ways of creating a tuple

```
1 >>>a = () #returns an empty tuple
2 >>>b = tuple() #built-in constructor method
3 >>>c = 4, #using trailing comma
4 >>>d = (4,) #using singleton object
5 >>>e = tuple([1,2]) #passing an iterable object
6 >>>f = 1,2,3 #separating items with comma
7 >>>g = (1,2,3) #separating items with comma
```

Constructor `tuple()` can take maximum one argument and creates a tuple of an iterable object of same items and same order. Without comma at line 3 and 4, Python would consider them as single-expression statement. At line 6 and 7, comma delimit individual elements within the parentheses, forming tuples with multiple elements. Each comma acts as a separator, indicating that the values on either side of it are distinct items in the tuple.

Binary Sequence Types – bytes and bytearray

Both bytes and bytearrays can be used to store binary data such as images, audio files, network packets, or serialized objects. They can also be used in cryptography, network communication, and low-level system programming. Some of the operations that can be performed on bytes and bytearrays include slicing, concatenation, decoding, encoding, hashing, and encryption.

Bytes are represented by the bytes type and can be created using the `'b'` prefix before a string literal or by using the `bytes()` constructor. Once created, a bytes object cannot be modified in place. Some examples of bytes literals given below:

```

1 >>>b'hello'
2 b'hello'
3 >>>my_bytes = bytes([0x48, 0x65, 0x6c, 0x6c, 0x6f])
4 >>>print(my_bytes)
5 b'Hello'
6 >>>print(type(my_bytes))
7 <class 'bytes'>

```

In the above example, a bytes object named `my_bytes` is created using the `bytes()` constructor and passing a list of integers representing the ASCII codes in hexa-decimal form for the characters 'H', 'e', 'l', 'l', and 'o'. The resulting bytes object is equivalent to the bytes literal `b'Hello'`. A bytes object can be created from a Unicode string using UTF-8 encoding as follows:

```

my_str = 'unicode string'
my_bytes = my_str.encode('utf-8')
# print the bytes object and its type
print(my_bytes) # prints b'unicode string'
print(type(my_bytes)) # prints <class 'bytes'>

```

The bytearray provides a way to represent sequences of bytes in a more flexible and efficient way than strings or lists of integers. The working of bytearray is shown in code Listing 3. 11 and converted to a string using the `str()` function. Like other sequence types, bytearray objects also support indexing.

Listing 3. 11 Creating and manipulating a bytearray object

```

# Create a byte array
>>>data = bytearray(b'\x00\x01\x02\x03\x04\x05\x06\x07')
# Modify the data in place
>>>data[2:5] = b'\xff\xff\xff'
# Convert the byte array to a string and print it
>>>print(str(data))
bytearray(b'\x00\x01\xff\xff\xff\x05\x06\x07')

```

Some of its purposes and uses are listed below:

- Efficient storage of binary data: Byte arrays provide a compact and efficient way to store binary data, such as image or audio files, network packets, or serialized objects. Because they are mutable, you can modify the data in place without having to create a new copy of the entire sequence.
- Network communication: Byte arrays can be used to represent messages or packets in network communication protocols, such as TCP, UDP, or HTTP. They can be sent and received over a network connection using socket programming, and they can be parsed and processed on the receiving end.
- Cryptography: Byte arrays can be used to represent cryptographic keys, digests, or ciphertexts. They can be processed using cryptographic algorithms, such as encryption, decryption, or hashing, and they can be compared for equality or used as inputs to other cryptographic operations.
- Low-level system programming: Byte arrays can be used to interact with low-level system APIs that require binary data, such as device drivers, file I/O, or memory-mapped files. They can be

passed as arguments to C functions using the ctypes module or accessed directly using the memory view object.

Dictionary

A dictionary is a mapping object which maps hashable (means an object if it has a hash value which never changes during the lifetime) values. It is an unordered collection of data values, and all items are stored in a *key: value* pair. A single key is associated with a value, and it is a mutable type of object. Two ways an empty dictionary can be formed either using a pair of curly braces ({}), or by calling constructor method *dict()*:

```
>>>d1 = {}
>>>d2 = dict()
>>>type(d1)
<class 'dict'>
```

By passing *key: value* pair inside a pair of curly braces ({}), a dictionary can be formed and keep in mind dictionary keys are always immutable type, means non-modifiable objects are allowed as dictionary key.

```
>>>d = {'lan': 'Python', 'ver': 3.11}
>>>d
{'lan': 'Python', 'ver': 3.11}
```

An identifier of immutable type can be used as key-value pair to form a dictionary. The list [(*'lan'*, *'Python'*), (*'ver'*, 3.11)] contains tuples, where each tuple represents a key-value pair. can also be created by passing a list of tuples.

```
>>> dict([('lan', 'Python'), ('ver', 3.11)])
{'lan': 'Python', 'ver': 3.11}
```

Dictionary supports searching an item in arbitrary order and all keys in dictionary should be unique. On adding a new item with existing key can update its value only:

```
>>>d['lan'] = 'c' #updating the value of key lan to c
>>>d['ver'] = 'c11' #updating the value of key ver to c11
>>>d
{'lan': 'c', 'ver': 'c11'}
```

Python dictionaries also support copy from another dictionary, removing a specific item based on key or pop out the last *key: value* pair item. All these operations with creation of dictionary from other data types discussed in Chapter 7.

Sets

A set object is an unordered collection of distinct hashable objects. Common uses include membership testing, removing duplicates from a sequence, and computing mathematical operations such as intersection, union, difference, and symmetric difference.

Python has two built-in set types as **set** and **frozenset**. Out of these set is mutable type means items can be altered later and frozenset is immutable type, can be used as a key of a dictionary. Constructor methods like other built-in types can be used to create any of this type:

```
>>>s1 = set()
>>>s2 = frozenset()
```

Remember using a pair of empty curly braces returns a dictionary object instead of a set. Non-empty sets (not frozensets) can be created by placing a comma-separated list of elements within braces, in addition to the set constructor as shown below:

Listing 3.12 Examples of valid set creation

```
>>>s3 = {1,}
>>>s4 = {1}
>>>s5 = {4,4.3}
>>>s6 = set({1,2,3,6.7,'w',(1,2)})
```

Inside the set and frozenset constructor, items should be of a hashable type. In general, all immutable types listed in Table 3.6 can be used to create a set or frozenset. A dictionary type can also be used to create any of these types, only keys are considered as items of the set.

```
>>>set({3:2})
{3}
>>>frozenset({3:2})
frozenset({3})
```

The discussion of sets concludes here. The remaining concepts and operations will be covered in Chapter 7.

3.4.4 User-defined Type

The purpose of a user-defined type is to create a custom data type that meets the specific needs of a particular application or domain. By defining a new type, you can encapsulate data and its behaviour into a single unit, which makes your code more organized, modular, and reusable. A user defined type has specific need and advantages over other data types discussed in previous sections. In python a user-defined data type can be created using the keyword **class**. Here's an example of a simple class creation without any body:

```
>>> class Sample:
...     pass
```

The indentation is necessary to start with the body of a class and the pass statement in the above class declaration represents an empty class. This class hasn't done anything yet. We can create an object of this class by calling its constructor:

```
>>> obj = Sample()
>>> obj
<__main__.Sample object at 0x000002043F3DF810>
```

In the output, `<__main__.Sample>` indicates that the `Sample` class is defined in the current Python module (`__main__`), signifies that `obj` is an object at memory address `0x000002043F3DF810`. We mostly interact with objects through methods and attributes, which is discussed in the Second part of the book.

3.4.5 None Type

Python doesn't have NULL keyword like other computer languages, but there is a **None** type. It is the

function's return value that “doesn't return anything”. `None` is often used to represent the absence of a value, as default parameters are not passed to the function. You can't assign a `NULL` value to the variable; if you do, it is illegal, and it will raise a **`NameError`**. Some important points related to `None` value are given below:

- Comparing `None` to anything will always return `False` except `None` itself.
- `None` is not a `0`.
- `None` is not an empty string.
- `None` is not the same as `False`.

In Python `None` is a special keyword that signifies the absence of value. It's used to indicate that a variable doesn't hold any data or that a function doesn't return anything explicitly.

```
>>> type(var)
<class 'NoneType'>
>>> if var is None:
...     var = 3
...
>>> var
3
```

3.5 Object Types

Python data types discussed in the previous section can be categorized into two groups based on, “how objects need to be handled during their lifetime”? Such as mutable and immutable and categorization of each object are listed in Table 3.6.

Table 3.6 Categorization of Object Types

Class	Explanation	Type
<code>bool</code>	boolean value	Immutable
<code>int</code>	integer value	Immutable
<code>float</code>	floating point value	Immutable
<code>decimal</code>	<code>decimal.Decimal</code> value	immutable
<code>fraction</code>	<code>fractions.Fraction</code> value	Immutable
<code>complex</code>	Numbers with real and imaginary part	Immutable
<code>list</code>	sequence of objects of mutable nature	Mutable
<code>tuple</code>	sequence of objects of immutable nature	Immutable
<code>range</code>	List of values in a range	Immutable
<code>str</code>	character/ string	Immutable
<code>set</code>	set of distinct objects that are of unordered nature	Mutable
<code>frozenset</code>	set class of immutable nature	Immutable
<code>dict</code>	dictionary or associative mapping	Mutable
<code>bytes</code>	Used to store and manipulate binary data	Immutable
<code>bytearray</code>	Used to store and manipulate binary data	Mutable

3.5.1 Mutability

Mutable in Python can be defined as the object that can change or be regarded as something changeable in nature. In Python mutable objects means the ability to modify the value of any object. In Listing 3.13, memory address of the list `my_list`, is printed using built-in function `id()`, i.e., 3048260948800. Later in the next line, 0th index item of `my_list` is updated to 10 without any error. It means a list object is mutable by nature and its id also remains same.

Listing 3.13 Mutability testing of list type

```
>>> my_list = [1, 2, 3]
>>> id(my_list)
3048260948800
>>> my_list[0] = 10
>>> id(my_list)
3048260948800
```

3.5.2 Immutability

Immutable objects in Python can be defined as objects that do not change their values and attributes over time. These objects become permanent once created and initialized. Python has numbers, tuples, strings, frozen sets, and user-defined classes with some exceptions. Values of these types can't be changed and remains permanent once they are initialized and hence called immutable. Once modifying different memory block is refereed by the name and this can be verified using `id()`. In Listing 3.14, immutability has been tested over string object `str1`, and after setting to new value 'Python', its memory address is changed. Means after modifying `str1` start referring to a new memory block.

Listing 3.14 Immutability testing of string type

```
>>> str1 = 'Hello'
>>> id(str1)
3048259007088
>>> str1 = 'Python'
>>> id(str1)
140715314864976
```

3.6 Variable Assignments

We have already seen couple of examples of variable assignments, a statement having an equal sign (=). Variables are symbolic names in any programming and in this section, we will learn various types of assignments based on data. Like others, in Python, variables need not be declared or defined in advance. To create a variable, you just assign it a value and then start using it.

```
>>>a = 4
```

This statement can be interpreted as “a is assigned the value 4.” Once this is done, `a` can be used in any statement or expression, and its value will be substituted, as shown below:

```
>>>a+4
8
```

In the last expression the value of **a**, is substituted and added with 4 and gets the result as 8. It is also possible to update the value of **a**, or can be used to assign to a new variable **b**, as shown below:

```
>>>a = 10 #updating the value of a
>>>a
10
>>>b = a #assigned to b
>>>b
10
```

Python also allows chained assignment, which makes it possible to assign the same value to several variables simultaneously:

```
>>>a = b = c = 5
>>>a
5
>>>b
5
```

The chained assignment in the above example, assigns 5 to the variables **a**, **b**, and **c** simultaneously. It is possible to assign multiple values to different variables at the same time. Python assigns values from right to left. When assigning multiple variables in a single line, different variable names are provided to the left of the assignment operator separated by a comma. The same goes for their respective values except they should be the right of the assignment operator.

```
>>>a, b = 4, 8
>>>a
4
>>>b
8
```

When using this method to declare variables, it's essential to pay attention to the order of variable names and their associated values. The first variable name on the left side of the assignment operator will receive the first value on the right side, and this pattern continues for subsequent variables. This approach is commonly referred to as multiple assignment. Importantly, the values being assigned do not have to be of the same data type; instead, they can consist of a mixture of valid Python data types.

```
>>>a, b, c, d = 4, "Python", 3.14, True
```

The above statement is an example of multiple variable assignment in a single line. It assigns values to four variables (**a**, **b**, **c**, and **d**) in a single statement. Each variable is assigned a corresponding value from the right-hand side of the statement.

3.7 Input and Output

3.7.1 Taking Input

Programs often need to obtain data from the user, usually by way of input from the keyboard. One way to accomplish this in Python with **input()** function and its general form is given below:

```
variable_name = input([prompt])
```

[prompt] is optional inside the parentheses of the **input()** function, you can include an optional

prompt message, which is a string enclosed in a single or double quotes. This prompt provides guidance or instructions to the user about what kind of input is expected. If you don't specify any prompt, the **input()** function will simply wait for the user to type something. Here's an example of how you might use this syntax:

```
>>>name = input("Please enter your name: ")
Please enter your name:
```

It is important to note that, **input()** function treats all user input as strings. If you need to work with numerical values, you'll need to convert the input to the appropriate data type using respective conversion functions such as **int()**, **float()**, after capturing it with **input()**.

```
>>>user_input = input('Enter a number.....')
Enter a number.....34
>>>print(type(user_input))
<class 'str'>
```

Next example demonstrates the conversion of a user input to integer. In this example, the user is prompted to enter their age. The input is captured as a string (**age_str**) and then converted to an integer using the **int()** function.

```
>>> age_str = input("Please enter your age: ")
Please enter your age: 34
>>> age = int(age_str)
>>> type(age)
<class 'int'>
```

In the next example, user is prompted to enter their weight in kilograms. The input is captured as a string (**weight_str**) and then converted to a floating-point number using the **float()** function.

```
>>> weight_str = input("Please enter your weight in kilograms: ")
Please enter your weight in kilograms: 74.5
>>> weight = float(weight_str)
>>> type(weight)
<class 'float'>
```

These examples demonstrate how to use the **input()** function to capture user input as strings and then convert them to different data types as needed for your program. Remember to handle input validation and errors to ensure that the user provides input in the expected format.

3.7.2. Printing Output

In Python, the **print()** function is used to display output to the standard output, typically the console or terminal. It allows you to output text, numbers, variables, and other data to the screen. The simplest form of **print()** function is to display a single string or variable's value:

```
>>>print("Hello, World!")
Hello, World!
```

The **print()** function can take multiple arguments, separated by commas. It will print each argument, separated by a space, and automatically add a newline character (**\n**) at the end by default:

```
>>>name = "Alice"
>>>age = 30
>>>print("Name:", name, "Age:", age)
Name: Alice Age: 30
```

Calling `print()` without arguments results in a blank line, which is a line comprised solely of the newline character, e.g. `print('\n')`. Don't confuse this, with an empty line (means zero characters), not even the newline e.g. `print('')`. Before passing a string object for printing, it can be formatted in many ways and all such methods are discussed in Section 5.9 of Chapter 5.

In Python 3, `print()` function uses two keyword arguments '`sep`' and '`end`' to modify the output string. While printing, `print()` function inserts a space between the values and that can be modified by using the keyword argument '`sep`'. See the listed example and in the output of second `print()`, comma(,) has been placed after every string literals.

```
>>> print('I am',37, 'Years old')
I am 37 Years old
>>> print('I am',37, 'Years old',sep=',')
I am,37,Years old
```

Separator keyword is very useful while formatting a date and based on our requirement we can pass as slash '/' or a hyphen '-' to separate day and month.

```
>>> print(12,11,2022,sep='/')
12/11/2022
>>> print(12,11,2022,sep='-')
12-11-2022
```

Unless the `end` keyword is used within print, it will always append a newline character ('\n'), at the end of what is being printed.

```
>>> print('A',end='');print('B');print('C')
AB
C
```

Due to use of `end` keyword within first print statement, output of the second print, placed in the same line as '`AB`'. But the third print statement prints `C` in a newline, because second print statement doesn't come with `end` keyword. Both keywords can be used together, that can help us to print data in a readable fashion.

```
>>> print('This','book','is','prepared','for', sep = '_', end='_Students')
This_book_is_prepared_for_Students
```

3.8 Built-in Functions

The Python interpreter has several functions and types built into it that are always available. These are listed in with a short description about their work. We would recommend go to the

link as given in the footnote of this page, to know more about this built-in²¹ functions.

Table 3.7 List of built-in functions

Function	Description
<code>abs(x)</code>	Return the absolute value of a number x. The argument may be an integer, a floating-point number. For complex number, functions return its magnitude.
<code>aiter(async_iterable)</code>	Return an asynchronous iterator for an asynchronous iterable.
<code>all(iterable)</code>	Returns True if all the elements of an iterable object are True
<code>any(iterable)</code>	Returns True if atleast one of the elements of an interable is True
<code>ascii(object)</code>	Return a string in printable from, but escaping the non-ASCII characters
<code>bin(n)</code>	Converts an integer number to its binary from with prefixed 0b.
<code>bool(x=False)</code>	Return a Boolean value, i.e. one of True or False.
<code>breakpoint()</code>	A built-in tool introduced in Python 3.7 that simplifies debugging by pausing your code's execution at a specific line.
<code>bytearray()</code>	Return a new array of byte.
<code>bytes()</code>	Return a new “bytes” object which is an immutable sequence of integers in the range $0 \leq x < 256$.
<code>callable(object)</code>	Returns True, if the object argument appears callable, False if not.
<code>chr(i)</code>	Return the string representing a character whose Unicode code point is the integer i. For example, <code>chr(97)</code> returns the string 'a'
<code>classmethod(anymethod)</code>	In Python 3, it is a decorator (<code>@classmethod</code>) instead of function and used to define a class method.
<code>compile()</code>	Compile the source into a code or AST object.
<code>complex()</code>	Returns a complex number
<code>delattr(object, name)</code>	Is used to dynamically delete an attribute from an object.
<code>dict()</code>	Creates a new dictionary
<code>dir()</code>	Without arguments, return the list of names in the current local scope.
<code>dir(object)</code>	With an argument, attempt to return a list of valid attributes for that object.
<code>divmod(a, b)</code>	Take two (non-complex) numbers as arguments and return a pair of numbers consisting of their quotient and remainder when using integer division.
<code>enumerate(iterable)</code>	Return an enumerate object. iterable must be a sequence
<code>eval(expression)</code>	The expression argument is parsed and evaluated as a Python expression
<code>exec(object)</code>	This function supports dynamic execution of Python code and object must be either a string or a code object.
<code>filter(function, iterable)</code>	Construct an iterator from those elements of iterable for which function is true.

²¹ <https://docs.python.org/3/library/functions.html>

<code>float()</code>	Return a floating-point number constructed from a number or string x.
<code>format(value, format_spec='')</code>	Convert a value to a “formatted” representation, as controlled by <code>format_spec</code> .
<code>frozenset()</code>	Returns a new frozenset object.
<code>getattr(object, name)</code>	Return the value of the named attribute of object. name must be a string.
<code>globals()</code>	Return the dictionary implementing the current module namespace.
<code>hasattr(object, name)</code>	The arguments are an object and a string. The result is True if the string is the name of one of the object’s attributes, False if not.
<code>hash(object)</code>	Return the hash value of the object (if it has one).
<code>help()</code>	Invoke the built-in help system.
<code>hex(x)</code>	Convert an integer number to a lowercase hexadecimal string
<code>id(object)</code>	Return the “identity” of an object.
<code>input()</code>	Used to take input from user.
<code>int()</code>	Return an integer object constructed from a number or string x
<code>isinstance(object, classinfo)</code>	Return True if the object argument is an instance of the classinfo argument.
<code>issubclass(class, classinfo)</code>	Return True if class is a subclass of classinfo.
<code>iter(object)</code>	Return an iterator object.
<code>len(s)</code>	Return the length (the number of items) of an object.
<code>list(object)</code>	Returns a list of the object, if it is a sequence type.
<code>locals()</code>	Update and return a dictionary representing the current local symbol table.
<code>map(function, iterable)</code>	Return an iterator that applies function to every item of iterable, yielding the results.
<code>max(iterable)</code>	Return the largest item in an iterable or the largest of two or more arguments.
<code>memoryview(object)</code>	Allows direct read and write access to the object's byte-oriented data, offering performance benefits and avoiding unnecessary copies.
<code>min(iterable)</code>	Return the smallest item in an iterable or the smallest of two or more arguments.
<code>next(iterator)</code>	Retrieve the next item from the iterator by calling its <code>__next__()</code> method.
<code>oct(x)</code>	Convert an integer number to an octal string prefixed with “0o”.
<code>open(file, mode)</code>	Open a file with the permission specified as mode.
<code>ord(c)</code>	Given a string representing one Unicode character, return an integer representing the Unicode code point of that character.
<code>pow(base, exp)</code>	Return base to the power exp.
<code>print(*objects)</code>	Print an object
<code>property()</code>	Return a property attribute.
<code>range(stop)</code>	Rather than being a function, range is actually an immutable sequence

	type.
<code>repr(object)</code>	Return a string containing a printable representation of an object.
<code>reversed(seq)</code>	Return a reverse iterator
<code>round(number, ndigits)</code>	Return number rounded to ndigits precision after the decimal point.
<code>set()</code>	Return a new set object
<code>setattr(object, name, value)</code>	The function assigns the value to the attribute, provided the object allows it.
<code>slice(stop)</code>	Return a slice object representing the set of indices specified by range(start, stop, step).
<code>sorted(iterable)</code>	Return a new sorted list from the items in iterable.
<code>staticmethod()</code>	In Python 3, it is a decorator <code>@staticmethod</code> , transform a method into a static method.
<code>str(object)</code>	Return a string version of an object.
<code>sum(iterable)</code>	Returns the sum of an iterable
<code>super()</code>	Allows you to access and call methods from a parent (super) class within a subclass (child class).
<code>tuple()</code>	Returns a tuple
<code>type(object)</code>	Returns the type of an object
<code>vars(object)</code>	Return the <code>__dict__</code> attribute for a module, class, instance, or any other object with a <code>__dict__</code> attribute.
<code>zip(*iterables)</code>	Iterate over several iterables in parallel, producing tuples with an item from each one.
<code>__import__()</code>	Invoke by import statement.

Key Takeaway's

- Keywords are reserved words in Python, and they are defined for specific purpose.
- It is possible to test a string as keyword or not by importing keyword module.
- Commenting your code is an important part of documentation and python provides commenting as single line or multi-line.
- In python some data types are comes with built-in and some of them we can create using class as user defined.
- `int()`, `float()`, `complex()`, `Decimal()`, `Fraction()` constructor methods are used to from a respective type.
- In Python Integers supports variables length precision, it depends on the available memory.
- Python uses a fixed number of bytes (8 bytes) to represent floats. Therefore, it can represent some numbers in binary approximately.
- It is necessary to import decimal module to create an object of Decimal type and pass the number with required digits as string.
- A factional type can be converted to floating-point, integer, string and complex type using built-in constructor methods.

- Converting a Fractional to Decimal using `Decimal()` will raise `TypeError`.
- Python supports mutable and immutable objects based on their memory referencing during the program execution.
- Lists, sets and dictionary are mutable type sequences, can be updated later.
- Frozenset, tuple and string objects can't be altered later and considered as immutable objects.

*****END OF CHAPTER*****